

Experiment – 5

1. **Aim:** To design a simple LSTM Network.
2. **Objectives:** Students should be familiarize with Learning Architectures and Frameworks using LSTM and CNN.
3. **Prerequisite:** Python basics
4. **Pre-Experiment Exercise:**

Theory:

Long Short Term Memory Network(LSTM). In concept, an LSTM recurrent unit tries to “remember” all the past knowledge that the network is seen so far and to “forget” irrelevant data. This is done by introducing different activation function layers called “gates” for different purposes. Each LSTM recurrent unit also maintains a vector called the Internal Cell State which conceptually describes the information that was chosen to be retained by the previous LSTM recurrent unit. A Long Short Term Memory Network consists of four different gates for different purposes as described below:-

Forget Gate(f): It determines to what extent to forget the previous data.

Input Gate(i): It determines the extent of information be written onto the Internal Cell State.

Input Modulation Gate(g): It is often considered as a sub-part of the input gate and much literature on LSTM’s does not even mention it and assume it is inside the Input gate. It is used to modulate the information that the Input gate will write onto the Internal State Cell by adding non-linearity to the information and making the information Zero-mean. This is done to reduce the learning time as Zero-mean input has faster convergence. Although this gate’s actions are less important than the others and are often treated as a finesse-providing concept, it is good practice to include this gate in the structure of the LSTM unit.

Output Gate(o): It determines what output(next Hidden State) to generate from the current Internal Cell State.

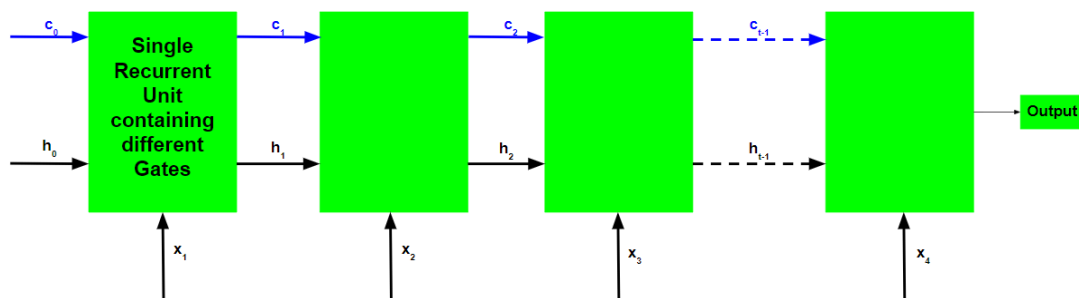


Fig:6.1 LSTM Network

CNN:

A convolutional neural network (CNN, or ConvNet) is a class of artificial neural network (ANN), most commonly applied to analyze visual imagery. CNNs are also known as Shift Invariant or Space Invariant Artificial Neural Networks (SIANN), based on the shared-weight architecture of the convolution kernels or filters that slide along input features and provide translation-equivariant responses known as feature maps. Counter-intuitively, most convolutional neural networks are not invariant to translation, due to the down sampling

operation they apply to the input. They have applications in image and video recognition, recommender systems, image classification, image segmentation, medical image analysis, natural language processing, brain–computer interfaces, and financial time series.

6. Laboratory Exercise

Procedure

- Take input the current input, the previous hidden state, and the previous internal cell state.
- Calculate the values of the four different gates by following the below steps:-
- For each gate, calculate the parameterized vectors for the current input and the previous hidden state by element-wise multiplication with the concerned vector with the respective weights for each gate.
- Apply the respective activation function for each gate element-wise on the parameterized vectors.
- Calculate the current internal cell state by first calculating the element-wise multiplication vector of the input gate and the input modulation gate, then calculate the element-wise multiplication vector of the forget gate and the previous internal cell state and then adding the two vectors.
- $c_{\{t\}} = i \odot g + f \odot c_{\{t-1\}}$

Post-Experiments Exercise:

A. Extended Theory:

- a. Explain Architecture of CNN.

B. Post Lab Program:

- a. Write a Python program to demonstrate any real life application using CNN .

C. Conclusion:

1. Write what was performed in the program (s) .
2. What is the significance of program and what Objective is achieved?

D. References:

- [1] <https://stackabuse.com/image-recognition-in-python-with-tensorflow-and-keras/>
[2] . <https://www.simplilearn.com/tutorials/deep-learning-tutorial/rnn>

Laboratory Exercise Procedure

Tanmay Bhatkar BE INFT A Roll No=>14 Batch 1

```
[1] import numpy as np
✓ Os

[2] # Activation functions
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def tanh(x):
    return np.tanh(x)
✓ Os

[3] # LSTM step function
def lstm_step(x_t, h_t_minus_1, c_t_minus_1, W_f, U_f, b_f, W_i, U_i, b_i, W_g, U_g, b_g, W_o, U_o, b_o):
    # Forget gate
    f_t = sigmoid(np.dot(W_f, x_t) + np.dot(U_f, h_t_minus_1) + b_f)

    # Input gate
    i_t = sigmoid(np.dot(W_i, x_t) + np.dot(U_i, h_t_minus_1) + b_i)

    # Input modulation gate (candidate cell state)
    g_t = tanh(np.dot(W_g, x_t) + np.dot(U_g, h_t_minus_1) + b_g)

    # Output gate
    o_t = sigmoid(np.dot(W_o, x_t) + np.dot(U_o, h_t_minus_1) + b_o)

    # Current cell state
    c_t = i_t * g_t + f_t * c_t_minus_1

    # Current hidden state
    h_t = o_t * tanh(c_t)

    return h_t, c_t, f_t, i_t, g_t, o_t

# Example input
x_t = np.array([0.5, -0.1]) # Current input (example)
h_t_minus_1 = np.array([0.1, 0.2]) # Previous hidden state (example)
c_t_minus_1 = np.array([0.3, -0.4]) # Previous cell state (example)

# Example weights and biases (random initialization)
np.random.seed(0) # For reproducibility

# Weights and biases for each gate (random values for demonstration)
W_f = np.random.randn(2, 2)
U_f = np.random.randn(2, 2)
b_f = np.random.randn(2)

W_i = np.random.randn(2, 2)
U_i = np.random.randn(2, 2)
b_i = np.random.randn(2)

W_g = np.random.randn(2, 2)
U_g = np.random.randn(2, 2)
b_g = np.random.randn(2)

W_o = np.random.randn(2, 2)
U_o = np.random.randn(2, 2)
b_o = np.random.randn(2)
```

The parameterized vectors for the current input and the previous hidden state by element-wise multiplication with the concerned vector with the respective weights for each gate.

OUTPUT:

```
# Perform the LSTM step
h_t, c_t, f_t, i_t, g_t, o_t = lstm_step(x_t, h_t_minus_1, c_t_minus_1, W_f, U_f, b_f, W_i, U_i, b_i, W_g, U_g, b_g, W_o, U_o, b_o)

# Print the results
print("Current hidden state (h_t):", h_t)
print("Current cell state (c_t):", c_t)
print("Forget gate (f_t):", f_t)
print("Input gate (i_t):", i_t)
print("Input modulation gate (g_t):", g_t)
print("Output gate (o_t):", o_t)

Current hidden state (h_t): [0.1114298 0.05401326]
Current cell state (c_t): [0.27658839 0.11954395]
Forget gate (f_t): [0.67482542 0.67713045]
Input gate (i_t): [0.58680752 0.40676183]
Input modulation gate (g_t): [0.12634596 0.95976588]
Output gate (o_t): [0.41309369 0.4539779 ]
```

Post Lab Program:

Write a Python program to demonstrate any real life application using CNN .

I am using a CNN trained on CIFAR-10 images dataset to detect images with labels airplane, automobiles,bird,cat,deer,dog,frog,horse,ship and truck. It has over 170 million samples.

```
!pip install tensorflow
```

```
# CNN for Image Classification (CIFAR-10 Example)

import tensorflow as tf
from tensorflow.keras import datasets, layers, models, regularizers
import matplotlib.pyplot as plt

# 1. Load and preprocess dataset
(x_train, y_train), (x_test, y_test) = datasets.cifar10.load_data()

# Normalize pixel values to [0,1]
x_train, x_test = x_train / 255.0, x_test / 255.0

# Class labels
class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']
```

```
↳ /usr/local/lib/python3.12/dist-packages/jax/_src/cloud_tpu_init.py:82: UserWarning: Tra
warnings.warn(
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ————— 6s 0us/step
```

```
# Detect TPU
try:
    tpu = tf.distribute.cluster_resolver.TPUClusterResolver()
    print("Running on TPU:", tpu.master())
except ValueError:
    tpu = None

# Initialize TPU strategy
if tpu:
    tf.config.experimental_connect_to_cluster(tpu)
    tf.tpu.experimental.initialize_tpu_system(tpu)
    strategy = tf.distribute.TPUStrategy(tpu)
else:
    strategy = tf.distribute.get_strategy() # fallback to CPU/GPU

print("Number of accelerators:", strategy.num_replicas_in_sync)
```

```
↳ Number of accelerators: 1
```

Using Google TPU on colab

```

▶ # Build CNN model with Dropout + L2 Regularization
model = models.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(32,32,3),
                  kernel_regularizer=regularizers.l2(0.001)),
    layers.MaxPooling2D((2,2)),
    layers.Dropout(0.25), # Dropout after conv block

    layers.Conv2D(64, (3,3), activation='relu',
                  kernel_regularizer=regularizers.l2(0.001)),
    layers.MaxPooling2D((2,2)),
    layers.Dropout(0.25),

    layers.Conv2D(64, (3,3), activation='relu',
                  kernel_regularizer=regularizers.l2(0.001)),
    layers.Flatten(),
    layers.Dense(64, activation='relu', kernel_regularizer=regularizers.l2(0.001)),
    layers.Dropout(0.5), # Dropout before output layer
    layers.Dense(10) # Output layer
])

```

Deciding the dropout and regularization rate

```

# 3. Compile model
model.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])

```

```

▶ # 4. Train model
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Data Augmentation
datagen = ImageDataGenerator(
    rotation_range=15,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True,
)

datagen.fit(x_train)

# Early stopping if val_loss doesn't improve for 5 epochs
early_stop = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)

# Reduce learning rate when val_loss doesn't improve for 3 epochs
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=3, min_lr=0.0001)

# Train with early stopping and learning rate reduction
history = model.fit(datagen.flow(x_train, y_train, batch_size=256),
                    epochs=25, # Increased epochs as per plan
                    validation_data=(x_test, y_test),
                    callbacks=[early_stop, reduce_lr])

```

Compiling and training model(Adding augmentations)

```

Epoch 1/25
391/391 — 28s 72ms/step - accuracy: 0.4743 - loss: 1.5790 - val_accuracy: 0.5513 - val_loss: 1.3404 - learning_rate: 0.0010
Epoch 2/25
391/391 — 28s 71ms/step - accuracy: 0.4942 - loss: 1.5202 - val_accuracy: 0.5579 - val_loss: 1.3132 - learning_rate: 0.0010
Epoch 3/25
391/391 — 28s 71ms/step - accuracy: 0.4987 - loss: 1.5072 - val_accuracy: 0.5823 - val_loss: 1.2770 - learning_rate: 0.0010
Epoch 4/25
391/391 — 28s 71ms/step - accuracy: 0.4995 - loss: 1.4907 - val_accuracy: 0.5733 - val_loss: 1.2845 - learning_rate: 0.0010
Epoch 5/25
391/391 — 28s 71ms/step - accuracy: 0.5116 - loss: 1.4785 - val_accuracy: 0.5832 - val_loss: 1.2762 - learning_rate: 0.0010
Epoch 6/25
391/391 — 28s 71ms/step - accuracy: 0.5149 - loss: 1.4749 - val_accuracy: 0.5906 - val_loss: 1.2471 - learning_rate: 0.0010
Epoch 7/25
391/391 — 28s 71ms/step - accuracy: 0.5242 - loss: 1.4592 - val_accuracy: 0.6129 - val_loss: 1.2232 - learning_rate: 0.0010

```

```

391/391 ————— 28s /ims/step - accuracy: 0.5711 - loss: 1.3559 - val_accuracy: 0.6472 - val_loss: 1.1209 - learning_rate: 0.0010
Epoch 20/25
391/391 ————— 28s 71ms/step - accuracy: 0.5779 - loss: 1.3574 - val_accuracy: 0.6614 - val_loss: 1.1023 - learning_rate: 0.0010
Epoch 21/25
391/391 ————— 28s 70ms/step - accuracy: 0.5723 - loss: 1.3670 - val_accuracy: 0.6565 - val_loss: 1.1102 - learning_rate: 0.0010
Epoch 22/25
391/391 ————— 27s 70ms/step - accuracy: 0.5729 - loss: 1.3642 - val_accuracy: 0.6697 - val_loss: 1.0781 - learning_rate: 0.0010
Epoch 23/25
391/391 ————— 28s 71ms/step - accuracy: 0.5852 - loss: 1.3407 - val_accuracy: 0.6614 - val_loss: 1.0988 - learning_rate: 0.0010
Epoch 24/25
391/391 ————— 27s 70ms/step - accuracy: 0.5842 - loss: 1.3387 - val_accuracy: 0.6574 - val_loss: 1.0990 - learning_rate: 0.0010
Epoch 25/25
391/391 ————— 27s 70ms/step - accuracy: 0.5866 - loss: 1.3249 - val_accuracy: 0.6601 - val_loss: 1.0879 - learning_rate: 0.0010

```

Few epochs while training

```

# 5. Evaluate model
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
print(f"\nTest accuracy: {test_acc:.2f}")

```

```

313/313 - 1s - 4ms/step - accuracy: 0.6697 - loss: 1.0781

Test accuracy: 0.67

```

```

# 6. Make predictions on some test images
probability_model = models.Sequential([model, layers.Softmax()])
predictions = probability_model.predict(x_test[:5])

```

```

1/1 ————— 0s 62ms/step

```

67 percent test accuracy

```

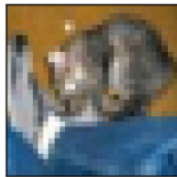
# Show first 5 test images with predicted labels
plt.figure(figsize=(10,5))
for i in range(5):
    plt.subplot(1,5,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.imshow(x_test[i])
    predicted_label = class_names[predictions[i].argmax()]
    true_label = class_names[int(y_test[i])]
    plt.xlabel(f"Pred: {predicted_label}\nTrue: {true_label}")
plt.show()

```

```

/tmp/ipython-input-1228871517.py:9: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated
true_label = class_names[int(y_test[i])]

```



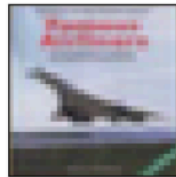
Pred: cat
True: cat



Pred: ship
True: ship



Pred: ship
True: ship



Pred: airplane
True: airplane



Pred: frog
True: frog

Examples of few predicted images

```

from tensorflow.keras.preprocessing import image
import numpy as np

def predict_custom_image():
    path = input("Enter the path of the image file: ")

    # Load and preprocess image
    img = image.load_img(path, target_size=(32, 32)) # CIFAR-10 uses 32x32
    img_array = image.img_to_array(img) / 255.0
    img_array = np.expand_dims(img_array, axis=0) # batch dimension

    # Prediction
    predictions = probability_model.predict(img_array)
    predicted_class = class_names[np.argmax(predictions)]

    # Show image with prediction
    plt.imshow(img)
    plt.axis("off")
    plt.title(f"Predicted: {predicted_class}")
    plt.show()

    print(f"Predicted Class: {predicted_class}")

```

Defining predict unseen image function

 `predict_custom_image()`

 Enter the path of the image file: `/content/airplane.jpg`
1/1  0s 23ms/step

Predicted: airplane



Predicted Class: airplane

Successfully predicted image of an airplane